



ROS 移动机器人的坐标系框架

ROS提供了一系列移动机器人开发的提升建议（ROS Enhancement Proposal，REP）。REP-105规定了移动机器人坐标系的位置、连接规范和连接维护组件。

坐标系位置

1. base_link

基坐标系，一般固定在移动机器人的基座上。为了让轮子贴合地面，还会使用base_footprint坐标系作为base_link的父坐标系。

2. odom

里程计坐标系，一般是一个固定在世界位置的坐标系，odom会随着时间变化而漂移，无法做为长期的全局坐标系使用。但机器人在odom坐标系下的位置是连续变化的，并不会跳跃，所以控制器计算速度时一般采用里程计位置作为当前位置来计算速度。

3. map

地图坐标系，固定在世界位置的坐标系，z轴向上，机器人在地图坐标系下姿态不会随着时间漂移，一般作为长期全局使用。但是map坐标系不是连续变化的，机器人在map坐标系中的姿态会随时发生跳跃变化。

4. earth

固定在地心位置，同时使用多个地图时，可以通过earth坐标系来将多个地图连接起来。

tf坐标关系

1. earth -> map

全球坐标到局部地图坐标

可选；室内/单车/单地图可以没有

2. map -> odom

全局定位对局部里程计的修正

通常动态；低频；可能跳变

3. odom -> base_link

局部里程计输出的车体位姿

动态；高频；连续；可漂移

4. base_link -> sensor/actuator

车体到传感器/执行机构的外参

刚性安装则静态；有关节则动态

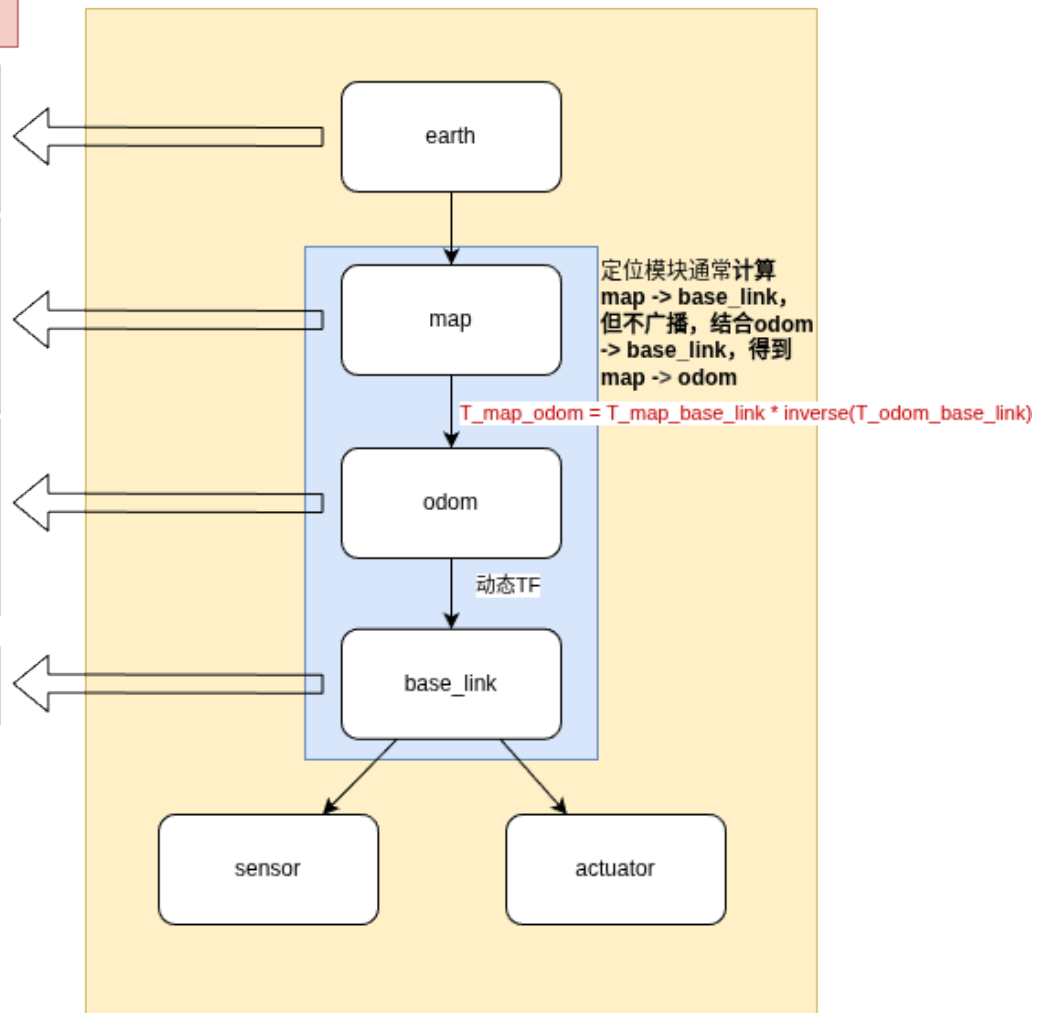
每个模块只需要声明自己的数据在哪个 frame 下，系统通过 TF 树自动换算到目标 frame。

地心地固坐标系。全球坐标到局部地图坐标。
多个机器人、多个地图、跨区域地图，初始化时没有可以后期接入。

来自全局定位、SLAM、AMCL、GNSS/INS 修正等，作用是修正 odom 的累计漂移。
长期全局参考，可以发生离散跳变，根据传感器观测不断重新计算机器人在地图中的位置

局部里程计坐标系，它通常在机器人启动或定位初始化时建立。它不是车上的某个点，而是一个“局部世界参考系”。来自轮速计、视觉里程计、IMU。
会随时间漂移，但短期准确且连续

一般表示机器人车体坐标系，是固定在机器人本体上的 frame



常见输出

局部定位算法的产出是odom -> base_link

全局定位算法计算map -> base_link但不广播，结合上面那个算出map->odom 然后广播出来。

foxglove 消息要填什么

以下为foxglove.pointcloud消息的字段：

```

struct PointCloud {
    /// @brief Timestamp of point cloud
    std::optional<Timestamp> timestamp;

    /// @brief Frame of reference
    std::string frame_id;

    /// @brief The origin of the point cloud relative to the frame of reference
    std::optional<Pose> pose;

    /// @brief Number of bytes between points in the `data`
    uint32_t point_stride = 0;

    /// @brief Fields in `data`. At least 2 coordinate fields from `x`, `y`, and `z` are required for each point's position; `red`, `green`, `blue`, and `alpha` are optional for customizing point's color.
    std::vector<PackedElementField> fields;

    /// @brief Point data, interpreted using `fields`
    std::vector<std::byte> data;

    /// @brief Encoded the PointCloud as protobuf to the provided buffer.
    ///
    /// On success, writes the serialized length to *encoded_len.
    /// If the provided buffer has insufficient capacity, writes the required capacity to *encoded_len and returns FoxgloveError::BufferTooShort.
    /// If the message cannot be encoded, writes the reason to stderr and returns FoxgloveError::EncodeError.
    ///
    /// @param ptr the destination buffer. must point to at least len valid bytes.
    /// @param len the length of the destination buffer.
    /// @param encoded_len where the serialized length or required capacity will be written
    FoxgloveError encode(uint8_t* ptr, size_t len, size_t* encoded_len);

    /// @brief Get the PointCloud schema.
    ///
    /// The schema data returned is statically allocated.
    static Schema schema();
};

```

- PointCloud 的 timestamp 是点云时间戳；
- frame_id 是参考坐标系；
- pose 是点云原点相对于该参考坐标系的位姿；
- point_stride 是相邻点之间的字节数；
- fields 描述 data 里的字段布局；
- data 是实际点数据。

然后还需要单独发布TF

base_link -> livox_frame

odom -> base_link

map -> odom